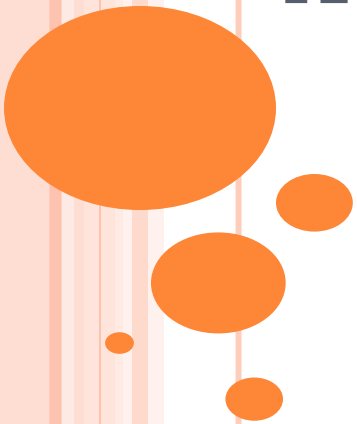
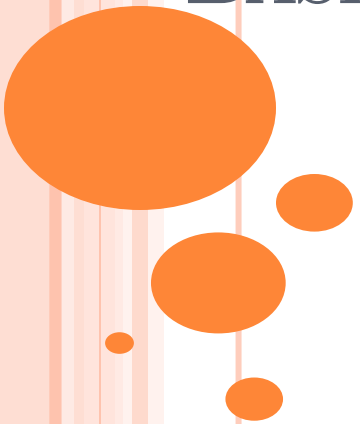
Acceso a bases de datos



Bases de datos relacionales



Bases de datos

- La mayor parte de las aplicaciones informáticas requieren en algún momento acceder a información almacenada en una base de datos.
- Distinguimos dos tipos de bases de datos:
  - Bases de datos relacionales. La información se organiza en tablas, donde cada tabla a su vez se organiza en filas y columnas. Se accede mediante SQL.
  - NoSQL. Bases de datos que no son relacionales. En ellas la información se organiza de forma muy diversa. Por ejemplo, están las bases de datos documentales,
- Python dispone de módulos para trabajar con las bases de datos más populares que se utilizan hoy en día, tanto relacionales como NoSQL

Antonio Martín Sierra

Instalación de módulos con pip

- Para acceder a bases de datos MySQL necesitamos el módulo mysql-connector. Es un módulo que no está incluido en la instalación por defecto y, por lo tanto, tendremos que instalarlo en nuestro entorno virtual.

```
>pip install mysql-connector-python
```

- Para otras bases de datos, por ejemplo oracle:

```
>pip install oracledb
```

Antonio Martín Sierra

Conexión con base de datos

- **Una vez instalado, ya podemos hacer uso del módulo mysql-connector para manipular bases de datos MySQL: recuperar, insertar, eliminar y actualizar filas de tablas.**
- **Para realizar cualquier operación, debemos establecer una conexión con la base de datos a través de la función *connect()* del módulo.**

```
import mysql.connector
db = mysql.connector.connect(
  host="localhost",
  user="root",
  password="root",
  port="3306",
  database="cursos"
)
```

- **Con ese objeto podemos obtener un cursor, que es el objeto que utilizaremos para operar contra la base de datos:**

```
cursordb=db.cursor()
```

Antonio Martín Ciorra

Recuperación de datos

- La instrucción SQL de tipo select permite recuperar una serie de datos de una tabla.
- El método *execute()* permite recorrer los datos de la consulta:

```
cursordb.execute("select * from cursos")
for f in cursordb:
        print(f)  #se imprimen los datos de cada fila en forma de tupla
```

- Se pueden definir condiciones en la instrucción de selección para recuperar filas que cumplan una condición determinada, y utilizar parámetros como valores de la condición:

```
values=(70,)
sql="select nombre,tematica from cursos where duracion>%s"
cursordb.execute(sql,values)
for tb in cursordb:
   print(tb)     #('Java', 'Programación')
                      #('Python', 'Programación')
                      #('Android', 'Programación')
db.close()
```

Antonio Martín Sierra

Inserción de filas

- La instrucción SQL de tipo insert se emplea para añadir nuevas filas a una tabla de la base de datos.
- La instrucción sql se proporciona como parámetro del método execute() del cursor. Cuando se ejecuta una consulta de acción se inicia una transacción que debe ser confirmada explícitamente antes de cerrar la conexión con la base de datos. Para confirmar una transacción se utiliza el método *commit()*

```
sql="insert into cursos(codCurso,nombre,tematica,duracion,precio) values(%s,%s,%s,%s,%s)"
values=(30,"HTML","Web",20,60)
cursordb.execute(sql,values)
db.commit() #confirmación de transacción
db.close()
```

Antonio Martín Sierra

Inserción de filas

- **Se pueden insertar varias filas proporcionando una lista de tupla de valores:**

```
sql="insert into cursos(codCurso,nombre,tematica,duracion,precio) values(%s,%s,%s,%s,%s)"
values=[(30,"HTML","Web",20,60),(40,"JavaScript","Web",40,90),(50,"Java","Programación",80,160)]
cursordb.execute(sql,values)
db.commit() #confirmación de transacción
db.close()
```

Antonio Martínez Sierra

Eliminar y actualizar filas

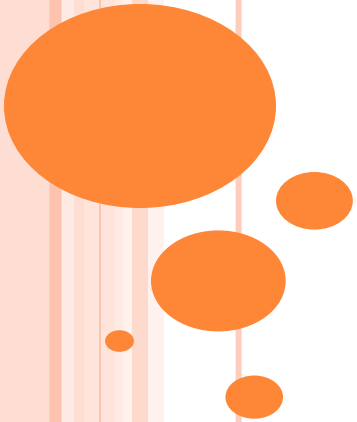
- **Al igual que insert, se trata de dos operaciones de acción que no devuelven resultados en el cursor**
- **Se llevan a cabo con las instrucciones SQL delete y update:**

```
sql="delete from cursos where codCurso=30"
cursordb.execute(sql)
print("Cursos eliminados: ", cursordb.rowcount)
sql="update cursos set precio=precio*%s where tematica=%s"
values=(1.05,"Programación")
cursordb.execute(sql,values)
print("Cursos actualizados en precio: ", cursordb.rowcount)
db.commit()
dc.close()
```

Permite conocer el número de filas afectadas

Antonio Espín Sierra

Acceso a MongoDb



Fundamentos

- **Se trata de una base de datos documental, donde la unidad de información es el documento (JSON).**
- **Los documentos a su vez se agrupan en colecciones**
- **Se puede descargar, junto con la herramienta compass:**

https://www.mongodb.com/download-center/community

Antonio Martín Sierra

El módulo pymongo

- Para acceder a bases de datos MongoDB desde Python utilizaremos el módulo pymongo que se instala con pip.
- Para para realizar cualquier operación debemos establecer una conexión con la base de datos, operación que realizaremos mediante el método MongoClient() de pymongo

```
import pymongo
uri="mongodb://localhost:27017"
serv=pymongo.MongoClient(uri)
```

- Para obtener una referencia a la colección:

```
db=serv.libreria
libs=db.libros
```

- La conexión se cierra con close()

```
serv.close()
```

Antonio Martín Sierra

Recuperar documentos

- **Con el método *find()* realizamos búsquedas de documentos en la colección. Con *find_one()* obtenemos únicamente el primer elemento del conjunto que cumpla la condición.**

- **Para realizar la búsqueda de documentos en MongoDb utilizamos condiciones expresadas a través de un filtro con formato JSON del tipo:**

```
{propiedad:{operador:valor}}
```

- **Por ejemplo:**

```
for l in libs.find():
    print(l)  #muestra todos los libros de la colección
for l in libs.find({"precio":{"$lt":25}}):
    print(l)  #muestra todos los libros de la colección cuyo precio sea inferior a 25
for l in libs.find({"titulo":{"$regex":"J"}})
    print(l)  #muestra todos los libros cuyo título contenga una J
```

Ignacio Martín Sierra

Inserción de documentos

- **A través de la referencia a la colección podemos añadir uno o varios documentos a la misma mediante los siguientes métodos:**
  - **insert_one(): Añade a la colección el documento:**

```
libs.insert_one({"titulo":"Python básico","autor":"varios","editorial":"Anaya","precio":19.5}
```

  - **insert_many(): Añade una lista de documentos:**

```
libs.insert_many([{"titulo":"Python básico","autor":"varios","editorial":"Anaya","precio":19.5},
{"titulo":"Django","autor":"Ramos","editorial":"Anaya","precio":21.8}
{"titulo":"Java 21","autor":"Martens","editorial":"Marte","precio":22.9}]
```

- **Cuando se añade a la colección, se incluye una propiedad id cuyo valor es generado de forma automática:**

```
{ "_id": {    "$oid": "6005589124b730549873ea5a"  },
"titulo": "Java 21",
"autor": "Martens",
"precio": 22.9}
```

Eliminación y actualización

- **Las colecciones proporcionan los siguientes métodos para eliminar documentos:**
  - **delete_one(): Elimina el primer documento que cumple la condición:**

```
libs.delete_one({"autor":"varios"} #elimina el primer libro cuyo autor es "varios"
```

  - **delete_many(): Elimina todos los documentos que cumplen la condición:**

```
libs.delete_many({"precio":{"$gte":50}}) #elimina todos los libros cuyo precio es superior o igual a 50
```

- **Disponemos de los siguientes métodos en una colección para actualizar documentos:**
  - **update_one(). Aplica la modificación al primer documento que cumple con la condición**

```
libs.update_one({"precio":{"$gte":50}},{"$dec":{"precio":10}}) #resta 10 al primer libro con precio >= 50
```

  - **update_many(). Aplica la modificación a todos los que la cumplen**

Antonio Martín Sierra